

RTB: Tecnologie PoC

Capitolato C6 — Second Brain

The Bitless | Gruppo 18

Ingegneria del Software, A.A. 2025/2026

Committente: Zucchetti S.p.A.

1 luglio 2026



Obiettivi del capitolato

COSA CHIEDE

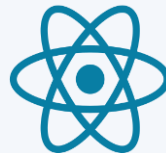
Un editor Markdown a due pannelli (testo → anteprima live), in cui il modello LLM opera sul testo.

FUNZIONI RICHIESTE

- Riassunto
- Riscrittura (variazione stilistica/correttezza grammaticale)
- Traduzione
- Sei Cappelli di De Bono (6 prospettive critiche)
- Distant Writing: l'utente detta l'intento, l'LLM scrive
- Salvataggio note su file (obbligatorio) e su DB opzionale



Frontend: React



Linguaggio: **TypeScript** vs JavaScript; tipi statici, meno bug, refactor più sicuri.

React

Cos'è

Una libreria per costruire interfacce a componenti.

Perché l'abbiamo scelto

- Scegliamo noi gli strumenti → un po' più di setup, ma voluto
- Ecosistema il più maturo → le librerie ci sono già e collaudate
- Sicurezza: mostra il Markdown senza inserire HTML grezzo

Vue

Cos'è

Un framework frontend a componenti.

Perché non l'abbiamo scelto

- componenti accessibili solo come copie della community, meno mature
- il modo più comune di mostrare il Markdown inietta HTML grezzo

Svelte

Cos'è

Un framework a compilatore: niente Virtual DOM, bundle piccoli e ottime prestazioni.

Perché non l'abbiamo scelto

- ecosistema più giovane
- le librerie che ci servono sono nate per React

React + ecosistema

Zustand

react-markdown

Tailwind + shadcn

Vite

Editor Markdown: CodeMirror 6



CodeMirror 6

Cos'è

Un editor di testo per il web, modulare: si includono solo le parti che servono.

Perché l'abbiamo scelto

- leggero (~100 KB): undo/redo, evidenziazione, ricerca e accessibilità già pronti
- resta stabile mentre l'anteprima si aggiorna in streaming

Monaco

Cos'è

Il motore di Visual Studio Code: un editor pensato per un IDE completo.

Perché non l'abbiamo scelto

- troppo pesante per delle semplici note
- tante funzioni da IDE che a noi non servono

Ace

Cos'è

Un editor di codice per il web, storico e molto usato in passato.

Perché non l'abbiamo scelto

- più datato e meno modulare di CodeMirror
- si integra peggio con React e TypeScript

Linguaggio Backend: Python



Python

Cos'è

Linguaggio con async/await nativo; è il linguaggio del backend.

Perché l'abbiamo scelto

- ecosistema AI/LLM il più maturo — è il cuore del prodotto
- carico I/O-bound → il GIL non pesa · supporto fino a ott 2028

Node.js + TS

Cos'è

JavaScript lato server: un solo linguaggio su frontend e backend.

Perché non l'abbiamo scelto

- l'ecosistema AI/LLM è meno ricco che in Python
- i tipi li allineiamo comunque generandoli dall'OpenAPI

Go

Cos'è

Linguaggio compilato, molto veloce e con ottima concorrenza.

Perché non l'abbiamo scelto

- il nostro carico è attesa (I/O), non calcolo: il vantaggio non emerge
- ecosistema AI più giovane

Framework Backend: FastAPI



FastAPI

Cos'è

Framework web Python basato sui tipi e su async. Genera da solo l'OpenAPI e valida con Pydantic.

Perché l'abbiamo scelto

- ecosistema AI/LLM più ampio di qualsiasi framework Python
- nato per l'async; validazione e documentazione già pronte

Flask

Cos'è

Un micro-framework Python minimale e storicamente sincrono.

Perché non l'abbiamo scelto

- per async, validazione e OpenAPI servono estensioni e lavoro manuale
- ciò che ci serve (streaming, contratti tipizzati) in FastAPI è già pronto

Litestar

Cos'è

Framework moderno come FastAPI (async, tipi, OpenAPI), più "tutto incluso".

Perché non l'abbiamo scelto

- community ed ecosistema più piccoli; il tooling AI/LLM è pensato per FastAPI
- il suo forte è la velocità sotto alto carico: non è il nostro caso

LIBRERIE

Pydantic

httpx

LLMClient + LiteLLM

Testing

Vitest



Cos'è

Test runner per JS/TS, integrato con Vite (unit e integrazione).

Perché, e non Jest

- riusa la pipeline di Vite: una sola configurazione
- API quasi identica a Jest

Playwright



Cos'è

Test end-to-end che pilota browser reali simulando l'utente.

Perché, e non Cypress

- multi-browser in una sola esecuzione
- auto-waiting → meno test instabili

pytest



Cos'è

Framework di test per Python, con fixtures e tanti plugin.

Perché, e non unittest

- mock di LLMClient via Depends (iniezione delle dipendenze)
- obiettivo coverage ~80%

Infrastruttura: Docker Compose



Docker Compose

Cos'è

Avvia tutto lo stack (web + api) da un solo file di configurazione.

Perché l'abbiamo scelto

- docker compose up: stesso ambiente per tutti e per la CI
- container = namespaces + cgroups, non una virtual machine (condivide il kernel → leggero)

Podman

Cos'è

Alternativa a Docker, compatibile e senza daemon root.

Perché non l'abbiamo scelto

- valida, ma Docker ha più diffusione, documentazione e familiarità

Setup manuale

Cos'è

Installare a mano Python, Node e dipendenze su ogni macchina.

Perché non l'abbiamo scelto

- fragile e diverso fra le macchine ("a me funziona")
- niente ambiente riproducibile

Proof of Concept

Pipeline di streaming end-to-end validata su tre funzioni reali

FUNZIONI VALIDATE

- Riassunto del testo
- Genera da istruzioni (Distant Writing)
- Genera da link (estrae il contenuto, poi scrive)

Lunghezza regolabile: breve / medio / dettagliato

VERIFICATO

- ✓ streaming SSE fluido · Stop chiude tutta la catena
- ✓ errore provider → 503 sanitizzato, testo preservato
- ✓ URL non valido → 400 (funzione da link)
- ✓ chiave mai esposta · docker compose up da zero

